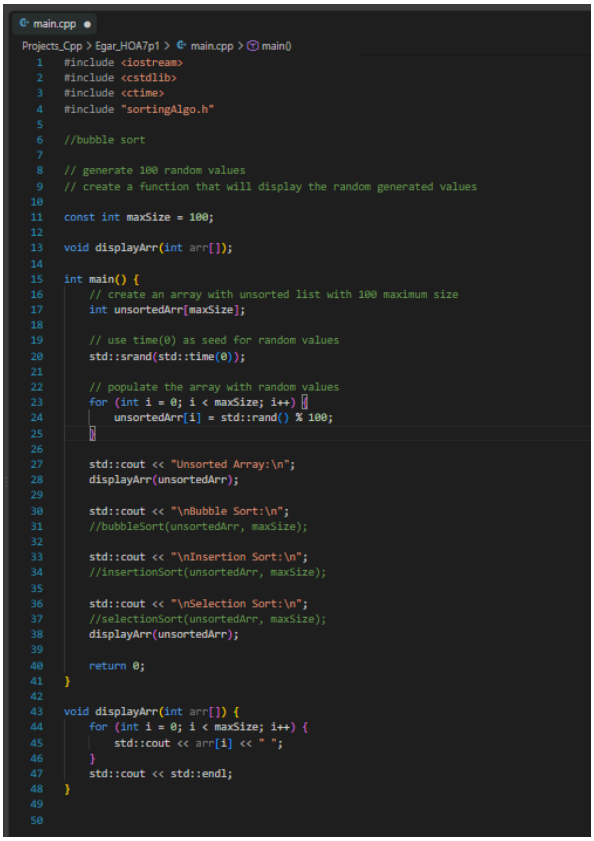
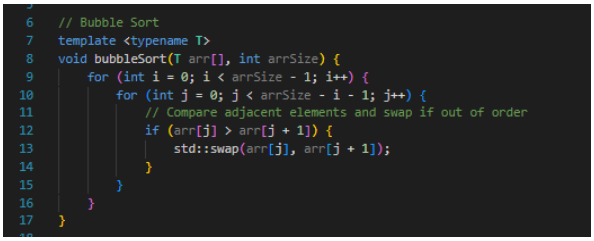
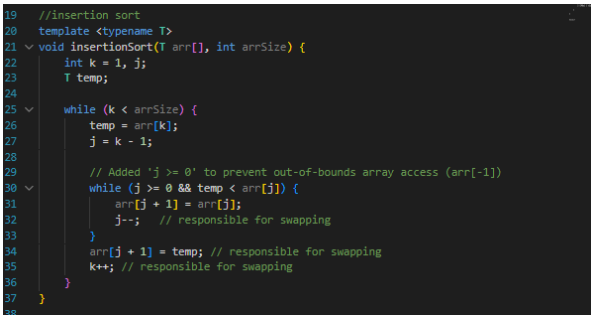
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	09/09/2026	**DATE SUBMITTED:**	09//2026
NAME:	Jarel Dave R. Egar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Screenshot | Main.cpp	
Screenshot | bubbleSort	
Screenshot | insertionSort	

Screenshot | selectionSort

```cpp
//helper function

template <typename T>
int routine_smallest(T A[], int maxSize, int k){ //int k = current index;

    int position, j;
    T smallestElement = A[k];

    for (int J = k+1; J < maxSize; J++) {
        if (A[J] < smallestElement) {
            smallestElement = A[J];
            position = J;
        }
    }

    return position;
}

//selection sort
template <typename T>
void selectionSort(T arr[], int arrSize) {
    int POS, temp;

    for (int i = 0; i < arrSize; i++) {
        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;

    }
}
```

Screenshot | sortingAlgo.h

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

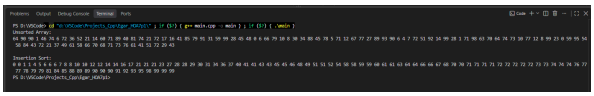
#include <iostream>

// Bubble Sort
template <typename T>
void bubbleSort(T arr[], int arrSize) {
    for (int i = 0; i < arrSize - 1; i++) {
        for (int j = 0; j < arrSize - i - 1; j++) {
            // Compare adjacent elements and swap if out of order
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

//insertion sort
template <typename T>
void insertionSort(T arr[], int arrSize) {
    int k = 1, j;
    T temp;

    while (k < arrSize) {
        temp = arr[k];
        j = k - 1;

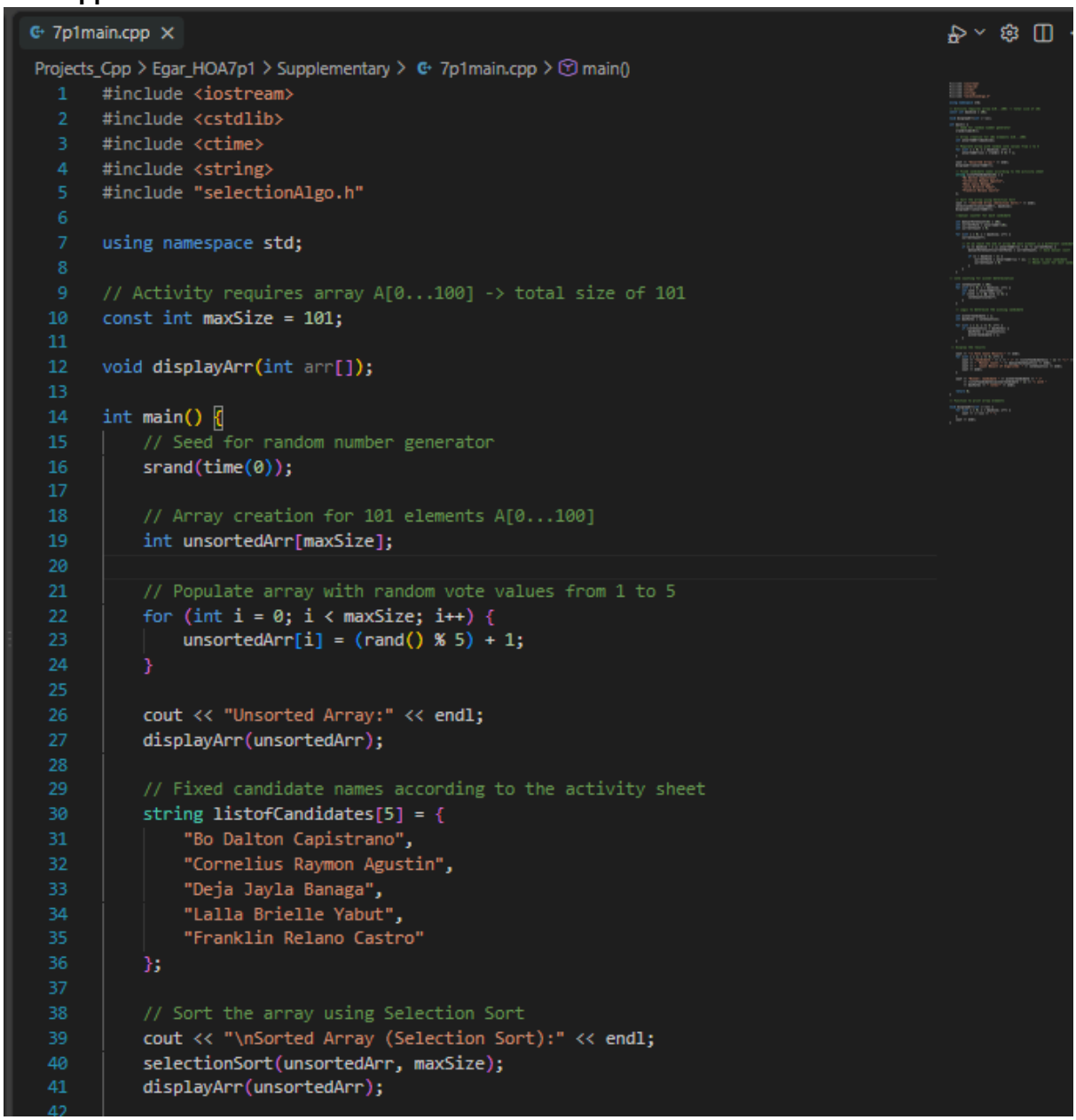
        // Added 'j >= 0' to prevent out-of-bounds array access (arr[-1])
        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;   // responsible for swapping
        }
        arr[j + 1] = temp; // responsible for swapping
        k++; // responsible for swapping
    }
}
```

	
Output | Bubblesort	
Output | InsertionSort	
Output | SelectionSort	

observation : Based on the procedure I learned how the 3 basic sorting algorithms are implemented and used. First for the bubble sort we created an array with the size to 100 that generates random values and then displays the unsorted numbers then uses bubble, insertion, and selection to sort the unsorted values. I also learned that these 3 has the same time complexity of O(n^2).

Supplementary activity:

Main.cpp




```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "selectionAlgo.h"

using namespace std;

// Activity requires array A[0...100] -> total size of 101
const int maxSize = 101;

void displayArr(int arr[]);

int main() {
    // Seed for random number generator
    srand(time(0));

    // Array creation for 101 elements A[0...100]
    int unsortedArr[maxSize];

    // Populate array with random vote values from 1 to 5
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = (rand() % 5) + 1;
    }

    cout << "Unsorted Array:" << endl;
    displayArr(unsortedArr);

    // Fixed candidate names according to the activity sheet
    string listofCandidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    // Sort the array using Selection Sort
    cout << "\nSorted Array (Selection Sort):" << endl;
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

```

```
    //manual counter for each candidate

    int manualVoteCount[6] = {0};
    int currentVote = unsortedArr[0];
    int currentCount = 0;

    for (int i = 0; i < maxSize; i++) {
        currentCount++;

        // If we reach the end of array OR next element is a different candidate
        if (i == maxSize - 1 || unsortedArr[i + 1] != currentVote) {
            manualVoteCount[currentVote] = currentCount; // Save manual count

            if (i < maxSize - 1) {
                currentVote = unsortedArr[i + 1]; // Move to next candidate
                currentCount = 0;                 // Reset count for next candidate
            }
        }
    }

// vote counting for winner determination

    int voteCount[6] = {0};
    for (int i = 0; i < maxSize; i++) {
        int vote = unsortedArr[i];
        if (vote >= 1 && vote <= 5) {
            voteCount[vote]++;
        }
    }

    // Logic to determine the winning candidate

    int winnerCandidate = 1;
    int maxVotes = voteCount[1];

    for (int i = 2; i <= 5; i++) {
        if (voteCount[i] > maxVotes) {
            maxVotes = voteCount[i];
            winnerCandidate = i;
        }
    }
```

```
84
85  // Display the results
86
87      cout << "\n Vote Count Results:" << endl;
88      for (int i = 1; i <= 5; i++) {
89          cout << "Candidate " << i << " (" << listofCandidates[i - 1] << "):" << endl;
90          cout << "  Manual Count: " << manualVoteCount[i] << endl;
91          cout << "  Count Result of Algorithm: " << voteCount[i] << endl;
92          cout << endl;
93      }
94
95      cout << "Winner: Candidate " << winnerCandidate << " ("
96           << listofCandidates[winnerCandidate - 1] << ") with "
97           << maxVotes << " votes!" << endl;
98
99      return 0;
100 }
101
102 // Function to print array elements
103
104 void displayArr(int arr[]) {
105     for (int i = 0; i < maxSize; i++) {
106         cout << arr[i] << " ";
107     }
108     cout << endl;
109 }
```

selectionSort.h

selectionAlgo.h

Projects_Cpp > Egar_HOA7p1 > Supplementary > selectionAlgo.h > routine_smallest<T>(T [], int, int)

```cpp
#ifndef SELECTIONALGO_H
#define SELECTIONALGO_H

// Helper function to find index of smallest element from index k onwards
template <typename T>
int routine_smallest(T A[], int maxSize, int k) {
    int position = k; // Default position to k in case A[k] is already smallest
    T smallestElement = A[k];

    for (int J = k + 1; J < maxSize; J++) {
        if (A[J] < smallestElement) {
            smallestElement = A[J];
            position = J;
        }
    }

    return position;
}

// Selection sort algorithm
template <typename T>
void selectionSort(T arr[], int arrSize) {
    for (int i = 0; i < arrSize - 1; i++) { // Optimization: stop at arrSize - 1
        int POS = routine_smallest(arr, arrSize, i);

        // Swap elements using template type T
        T temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}


#endif
```

OUTPUT:

```
PS D:\VSCode\Projects_Cpp\Egar_HOA7p1\Supplementary> cd "d:\VSCode\Projects_Cpp\Egar_HOA7p1\Supplementary\" ; if ($?) { g++ 7p1main.cpp -o 7p1main } ; if ($?) { .\7p1main }
Unsorted Array:
3 3 2 2 1 4 5 2 4 5 2 5 4 4 4 1 4 4 5 2 2 1 4 1 1 2 3 5 5 4 1 4 5 1 5 2 5 2 5 4 4 5 1 5 3 1 5 5 5 3 3 5 3 5 5 2 1 4 3 1 5 5 5 2 5 3 1 5 5 3 4 3 2 4 4 2 3 4 4 5 4 4 1 3 1 4 5 4 5 5 4 2 2 4 4 5 2 5 4 5 5

Sorted Array (Selection Sort):
1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

 Vote Count Results:
Candidate 1 (Ro Dalton Capistrano):
  Manual Count: 14
  Count Result of Algorithm: 14

Candidate 2 (Cornelius Raymon Agustin):
  Manual Count: 16
  Count Result of Algorithm: 16

Candidate 3 (Deja Jayla Banaga):
  Manual Count: 13
  Count Result of Algorithm: 13

Candidate 4 (Lalla Brielle Yabut):
  Manual Count: 26
  Count Result of Algorithm: 26

Candidate 5 (Franklin Relano Castro):
  Manual Count: 32
  Count Result of Algorithm: 32

Winner: Candidate 5 (Franklin Relano Castro) with 32 votes!
PS D:\VSCode\Projects_Cpp\Egar_HOA7p1\Supplementary>
```

Observation: I chose Selection Sort for this activity because it is simpler to implement and easy to trace for array operations. It has a time complexity of O(n), which works reliably for smaller datasets like this one.I reused the base code structure for the header file and function definitions, but updated main() to fulfill the new activity requirements. Specifically, I changed the modulo operation from 100 to 5 so the generated vote values will result in the 1 to 5 candidate range. Finally, I added the functions for manual vote counting and implemented the function to calculate the final vote totals and determine the winning candidate.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explaining how to start the code and what important syntaxesI must consider for this activity**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I asked Gemini to help me understand the code better for me to start it and asked it to tell me what are the syntaxes that I need, and I specify that it will not give me the code for the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.